



Université de Perpignan Via Domitia (UPVD)

UFR Sciences Exactes et Expérimentales - SEE

Département d'Informatique



TD

Deboggage, profiling, analyse de performances, optimisation

Thème

Debugging and Profiling

Réalisé par

— BOUZIDI Adel

2024/2025

Résumé du TD : Debugging et Profiling

Ce **TD** explore les **étapes essentielles** du développement, du débogage et de l'optimisation des performances d'un programme.

- **Exercice 1** : Installation du code source pour préparer l'environnement de travail.
- **Exercice 2** : Implémentation de la fonction `ssgemv` et utilisation des outils de débogage pour corriger les erreurs.
- **Exercice 3** : Mesure des performances du programme à l'aide de `time` et `rdtsc` pour analyser son temps d'exécution et son efficacité.
- **Exercice 4** : Utilisation des outils de **profiling** (`gprof` et `perf`) afin d'identifier les parties les plus consommatrices en ressources et orienter l'optimisation.

Ce TD permet ainsi d'acquérir des **méthodes rigoureuses de diagnostic et d'amélioration des performances**, essentielles pour le développement de logiciels fiables et efficaces.

Mots clés : GCC, GDB, gprof, Perf.

Table des matières

Table des matières	i
Table des figures	iii
Introduction	1
1 Exercice 1. Installation du code source	2
1.1 Introduction	2
1.2 Environnement de test	2
1.3 Installation et prérequis	3
1.4 Compilation du projet	4
1.5 Conclusion	5
2 Exercice 2. Codage et debugging	6
2.1 Introduction	6
2.2 Codage et Debugging	6
2.3 Conclusion	14
3 Exercice 3. Mesure de performance	15
3.1 Introduction	15
3.2 Comménçant par la machine virtuelle	15
3.2.1 Refaire l’Exercice 3 sur la nouvelle installation Ubuntu	19
3.3 Conclusion	20
4 Exercice 4. Profiling simple	21
4.1 Introduction	21
4.1.1 Résumé et interprétation des résultats gprof - Observations principales . .	23

4.1.2	Analysons les résultats	24
4.2	Conclusion	28
	Conclusion Générale	29

Table des figures

1.1	Le dossier prflib001.	3
1.2	La version de GCC.	4
1.3	La version de GDB.	4
1.4	La version de GPROF.	4
1.5	La version de PERF.	4
1.6	Compilation du projet.	5
2.1	Cblasl3.c Avant modification.	7
2.2	Implémentation de la fonction ssgemm dans cblasl3.cc.	8
2.3	Compilation.	8
2.4	Exécution du programme test001.exe.	9
2.5	Détection et correction de l'erreur avec GDB.	9
2.6	Détection et correction de l'erreur avec GDB - suite.	10
2.7	Détection et correction de l'erreur avec GDB avec l'option bt.	10
2.8	Inspection des Variables.	10
2.9	Vérification de l'Accès Mémoire.	11
2.10	Modification et Recompilation et test : (ligne 18 et 20).	11
2.11	Modification et Recompilation et test : (ligne 18 et 20) - suite.	12
2.12	Compilation avec les symboles de débogage.	13
2.13	Lancement du Valgrind sur le programme.	13
3.1	Exécution du programme avec time : time ./bin/test001.exe.	16
3.2	Modification du code de ssgemm pour ajouter la mesure avec rdtsc.	18
3.3	Compilation et exécution du programme.	18
3.4	Compilation du code source.	19

3.5	Exécution du programme avec time : (time ./bin/test001.exe).	19
3.6	Exécution du programme avec rdtsc pour mesurer les cycles CPU : (./bin/- test001.exe).	19
4.1	Les outils nécessaires.	21
4.2	Les résultats obtenus avec gprof - Le "Flat Profile".	22
4.3	Les résultats obtenus avec gprof -Le "Call Graph".	23
4.4	Modification apportée sur le script test001.cc : (remplacer 8 par 252).	24
4.5	Modification du Code pour Permettre un Profilage Efficace.	25
4.6	Recompilation du programme avec l'option -pg et exécution - Le "Flat Profile". .	25
4.7	Recompilation du programme avec l'option -pg et exécution - Le "Call Graph". .	26
4.8	Exécution et Analyse avec perf.	27

Introduction

L'optimisation et la correction des erreurs dans un programme informatique sont des étapes essentielles pour garantir un fonctionnement efficace et fiable. Ce TD porte sur deux aspects fondamentaux du développement logiciel : le débogage et le profiling (analyse de performance).

Dans un premier temps, je vais explorer l'installation et la mise en place du code source, suivies de l'implémentation d'une fonction clé, `ssgemm`, utilisée pour le produit matriciel. Ensuite, je verrai comment identifier et corriger les erreurs potentielles à l'aide d'outils de débogage. Une fois le programme fonctionnel, j'analyserai ses performances en mesurant le temps d'exécution et le nombre de cycles processeur nécessaires à son exécution. Enfin, j'utiliserai des outils de profiling, tels que `gprof` et `perf`, afin d'optimiser le programme en identifiant les goulots d'étranglement.

Chapitre 1

Exercice 1. Installation du code source

1.1 Introduction

Avant de pouvoir compiler, exécuter et analyser un programme, il est essentiel de mettre en place correctement son environnement de développement. Cet exercice a pour objectif de télécharger, extraire et organiser les fichiers sources nécessaires à l'exécution des prochains travaux.

Je vais travailler avec une archive compressée contenant le code source du projet. L'installation consistera à décompresser l'archive, organiser les fichiers et vérifier leur présence, afin d'assurer une base solide pour la suite des exercices de débogage et d'optimisation.

1.2 Environnement de test

J'ai réalisé le TD sur une machine physique avec Ubuntu. Voici les caractéristiques de l'environnement :

CPU :

```
root@N-5CG1328LRC:~# lscpu
Architecture:          x86_64
  CPU op-mode(s):      32-bit, 64-bit
  Address sizes:        36 bits physical, 48 bits virtual
  Byte Order:           Little Endian
CPU(s):                8
  On-line CPU(s) list:  0-7
Virtualization features:
  Virtualization:       VT-x
```



```
Hypervisor vendor:    Windows Subsystem for Linux
Virtualization type:  container
```

Mémoire :

```
root@N-5CG1328LRC:~# free -h
```

	total	used	free	shared	buff/cache	available
Mem:	15Gi	9.3Gi	6.3Gi	17Mi	223Mi	6.4Gi
Swap:	48Gi	257Mi	47Gi			

Système :

```
root@N-5CG1328LRC:~# lsb_release -a
```

```
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.5 LTS
Release:        22.04
```

1.3 Installation et prérequis

Après l'installation du code source, une arborescence de fichiers et de répertoires est générée dans le dossier du projet. Celle-ci contient les fichiers essentiels à la compilation, à l'exécution et aux tests du programme. Voici la structure obtenue après l'installation :

```
adel@adel:~/profiling/prflib001/prflast/prflib001$ ll
total 8
drwxrwx--- 1 root vboxsf 4096 mars 15 21:40 ./
drwxrwx--- 1 root vboxsf  0 mars 15 21:40 ../
drwxrwx--- 1 root vboxsf  0 févr. 3 23:54 bin/
drwxrwx--- 1 root vboxsf  0 févr. 3 23:54 build/
drwxrwx--- 1 root vboxsf  0 mars 15 21:40 include/
drwxrwx--- 1 root vboxsf  0 févr. 3 23:54 lib/
-rwxrwx--- 1 root vboxsf 2405 févr. 3 23:25 Makefile*
drwxrwx--- 1 root vboxsf  0 mars 15 21:40 src/
drwxrwx--- 1 root vboxsf  0 mars 15 21:40 tests/
```

FIGURE 1.1 – Le dossier prflib001.

Ensuite, une vérification des prérequis est essentielle pour garantir un environnement de développement complet. Le compilateur C GCC est déjà installé, permettant ainsi la compilation du code source en exécutable. Le débogueur GDB est également installé afin de faciliter l'analyse et la correction d'éventuelles erreurs dans le programme. Enfin, les outils de profiling, notamment gprof et perf, sont également installés pour analyser les performances du programme.

```
adel@adel:~/profiling/prflib001/prflib001$ gcc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

FIGURE 1.2 – La version de GCC.

```
adel@adel:~/profiling/prflib001/prflib001$ gdb --version
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

FIGURE 1.3 – La version de GDB.

```
adel@adel:~/profiling/prflib001/prflib001$ gprof --version
GNU gprof (GNU Binutils for Ubuntu) 2.38
Based on BSD gprof, copyright 1983 Regents of the University of California.
This program is free software. This program has absolutely no warranty.
```

FIGURE 1.4 – La version de GPROF.

```
adel@adel:~/profiling/prflib001/prflib001$ perf --version
perf version 6.8.12
```

FIGURE 1.5 – La version de PERF.

1.4 Compilation du projet

La compilation s'est déroulée avec succès, et tous les fichiers nécessaires à l'exécution et à l'utilisation du programme ont été générés. Voici les principaux fichiers produits :

- **bin/test001.exe** → Il s'agit de l'**exécutable de test**, permettant de vérifier le bon fonctionnement du programme après compilation. Ce fichier peut être exécuté directement pour tester les fonctionnalités implémentées.
- **lib/prflib.a** → Cette **bibliothèque statique** contient du code précompilé pouvant être réutilisé par d'autres programmes. Elle permet de regrouper des fonctions et des routines optimisées afin de faciliter la modularité et la réutilisation du code sans nécessiter un recompilage complet.
- **build/*.o** → Ces fichiers **objets** sont des résultats intermédiaires de la compilation. Chaque fichier **.o** correspond à un module compilé séparément à partir d'un fichier source **.c**, avant d'être lié pour produire l'exécutable final. Ils sont essentiels pour le processus d'édition de liens.

```

adel@adel:~/profiling/prflib001/prflast/prflib001$ make
g++ -Wall -I./include -std=c++17 -g src/cblasl0.cc -c -o build/cblasl0.o
g++ -Wall -I./include -std=c++17 -g src/cblasl1.cc -c -o build/cblasl1.o
g++ -Wall -I./include -std=c++17 -g src/cblasl3.cc -c -o build/cblasl3.o
ar -rc lib/prflib.a build/cblasl0.o build/cblasl1.o build/cblasl3.o
g++ -Wall -I./include -std=c++17 -g tests/test001.cc -c -o build/test001.o
g++ -Wall -I./include -std=c++17 -g build/test001.o lib/prflib.a -o bin/test001.exe
adel@adel:~/profiling/prflib001/prflast/prflib001$
adel@adel:~/profiling/prflib001/prflast/prflib001$
adel@adel:~/profiling/prflib001/prflast/prflib001$ ls
bin build include lib Makefile src tests
adel@adel:~/profiling/prflib001/prflast/prflib001$ ll bin/ build/ lib/
bin/:
total 64
drwxrwx--- 1 root vboxsf      0 mars  15 22:16 ./
drwxrwx--- 1 root vboxsf  4096 mars  15 21:40 ../
-rwxrwx--- 1 root vboxsf 59696 mars  15 22:16 test001.exe*

build/:
total 124
drwxrwx--- 1 root vboxsf      0 mars  15 22:16 ./
drwxrwx--- 1 root vboxsf  4096 mars  15 21:40 ../
-rwxrwx--- 1 root vboxsf 39880 mars  15 22:16 cblasl0.o*
-rwxrwx--- 1 root vboxsf  6152 mars  15 22:16 cblasl1.o*
-rwxrwx--- 1 root vboxsf 33136 mars  15 22:16 cblasl3.o*
-rwxrwx--- 1 root vboxsf 35088 mars  15 22:16 test001.o*

lib/:
total 84
drwxrwx--- 1 root vboxsf      0 mars  15 22:16 ./
drwxrwx--- 1 root vboxsf  4096 mars  15 21:40 ../
-rwxrwx--- 1 root vboxsf 79688 mars  15 22:16 prflib.a*

```

FIGURE 1.6 – Compilation du projet.

1.5 Conclusion

À l'issue de cet exercice, j'ai réussi à télécharger, extraire et organiser les fichiers sources du projet. Cette étape préliminaire est essentielle pour garantir un environnement de travail structuré et prêt pour la compilation, l'exécution et l'analyse du programme.

Dans ce qui suit, je présenterai la solution de l'exercice 2.

Exercice 2. Codage et debugging

2.1 Introduction

Cet exercice se concentre sur l'implémentation de la fonction `ssgemv`, une version simplifiée du produit matriciel, ainsi que sur l'utilisation d'outils de débogage pour identifier et corriger d'éventuelles erreurs.

L'objectif est double : écrire un code fonctionnel tout en adoptant une approche méthodique de débogage. En utilisant des outils comme GDB, j'apprendrai à analyser les erreurs, comprendre leur origine et appliquer des corrections pour garantir un programme stable et fiable.

2.2 Codage et Debugging

À présent, nous devons procéder à l'implémentation de la fonction `ssgemv` dans le fichier `cblas3.cc`. Cette fonction est essentielle pour effectuer des opérations de multiplication de matrices en utilisant des optimisations spécifiques aux bibliothèques BLAS (Basic Linear Algebra Subprograms).

Avant d'apporter les modifications nécessaires, voici le contenu actuel du fichier `cblas3.c`, qui servira de base à notre implémentation :

```

C: > Users > adelb > Documents > debo_profil_c > prflib001 > prflast > prflib001 > src > cblasl3.cc
1  #include <cblasl3.h>
2  #include <iostream>
3  /*
4   * Routines of cblas level 3
5   */
6
7  void ssgemm(
8      int m, int n, int k,
9      float alpha,
10     float ** A,
11     float ** B,
12     float beta,
13     float ** C)
14 {
15     // Implémenter ICI
16     std::cout << "*****" << std::endl;
17     std::cout << "  WARNING: Not Yet Implemented..." << std::endl;
18     std::cout << "*****" << std::endl;
19 }
20

```

FIGURE 2.1 – Cblasl3.c Avant modification.

Objectif de ssgemm :

La fonction `ssgemm` (Single-precision Scalar General Matrix Multiply) est une **version simplifiée de la multiplication de matrices** utilisée en algèbre linéaire et optimisée pour les calculs de haute performance. Elle fait partie des **opérations BLAS (Basic Linear Algebra Subprograms) de niveau 3**, qui sont spécialement conçues pour exploiter l'efficacité des architectures modernes, notamment via la parallélisation et l'utilisation de caches mémoire.

L'opération réalisée par `ssgemm` suit la **formule mathématique suivante** :

$$C = \beta C + \alpha(A \times B)$$

où :

- A est une matrice de dimensions (m, k) .
- B est une matrice de dimensions (k, n) .
- C est une matrice de dimensions (m, n) , qui stocke le résultat final.
- α et β sont des scalaires permettant de pondérer les contributions des matrices dans le calcul.

L'objectif principal de `ssgemm` est d'exécuter cette opération de manière **optimisée**, en réduisant le nombre d'accès mémoire et en maximisant l'utilisation des ressources du processeur, notamment via des techniques comme le **blocage de matrices** et SIMD.

Implémentation de ssgemm :

```
void ssgemm(
    int m, int n, int k,
    float alpha,
    float ** A,
    float ** B,
    float beta,
    float ** C)

{
    for (int i = 0; i <= m; ++i) {
        for (int j = 0; j <= n; ++j) {
            float sum = 0.0f;

            for (int p = 0; p < k; ++p) {
                sum += A[i][p] * B[p][j];
            }

            C[i][j] = alpha * sum + beta * C[i][j];
        }
    }

    // std::cout << "*****" << std::endl;
    // std::cout << "    WARNING: Not Yet Implemented..." << std::endl;
    // std::cout << "*****" << std::endl;
}
```

FIGURE 2.2 – Implémentation de la fonction ssgemm dans cblas3.cc.

Compilation et Tests du Projet

```
adel@adel:~/profiling/prflib001/prflast/prflib001$ make clean
rm -f ./build/*.o ./lib/*.a ./bin/*.exe src/*~ include/*~
adel@adel:~/profiling/prflib001/prflast/prflib001$ make
g++ -Wall -I./include -std=c++17 -g src/cblasl0.cc -c -o build/cblasl0.o
g++ -Wall -I./include -std=c++17 -g src/cblasl1.cc -c -o build/cblasl1.o
g++ -Wall -I./include -std=c++17 -g src/cblasl3.cc -c -o build/cblasl3.o
ar -rc lib/prflib.a build/cblasl0.o build/cblasl1.o build/cblasl3.o
g++ -Wall -I./include -std=c++17 -g tests/test001.cc -c -o build/test001.o
g++ -Wall -I./include -std=c++17 -g build/test001.o lib/prflib.a -o bin/test001.exe
adel@adel:~/profiling/prflib001/prflast/prflib001$
adel@adel:~/profiling/prflib001/prflast/prflib001$
```

FIGURE 2.3 – Compilation.

Exécution du programme test001.exe

Une fois la compilation terminée avec succès, je peux exécuter le programme de test test001.exe afin de vérifier son bon fonctionnement.

```

adel@adel:~/profiling/prflib001/prflast/prflib001$ ./bin/test001.exe

=====
Matrices before computations :
=====
[
  [ 1, 2, 3, 4, 5, 6, 7, 8]
  [ 9, 10, 11, 12, 13, 14, 15, 16]
  [ 17, 18, 19, 20, 21, 22, 23, 24]
  [ 25, 26, 27, 28, 29, 30, 31, 32]
  [ 33, 34, 35, 36, 37, 38, 39, 40]
  [ 41, 42, 43, 44, 45, 46, 47, 48]
  [ 49, 50, 51, 52, 53, 54, 55, 56]
  [ 57, 58, 59, 60, 61, 62, 63, 64]
]
[
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
]
[
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
]
Erreur de segmentation (core dumped)

```

FIGURE 2.4 – Exécution du programme test001.exe.

Détection et correction de l'erreur avec GDB

```

adel@adel:~/profiling/prflib001/prflast/prflib001$ gdb ./bin/test001.exe
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./bin/test001.exe...
(gdb) run
Starting program: /home/adel/profiling/prflib001/prflast/prflib001/bin/test001.exe
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

```

FIGURE 2.5 – Détection et correction de l'erreur avec GDB.

```

=====
Matrices before computations :
=====
[
  [ 1, 2, 3, 4, 5, 6, 7, 8]
  [ 9, 10, 11, 12, 13, 14, 15, 16]
  [ 17, 18, 19, 20, 21, 22, 23, 24]
  [ 25, 26, 27, 28, 29, 30, 31, 32]
  [ 33, 34, 35, 36, 37, 38, 39, 40]
  [ 41, 42, 43, 44, 45, 46, 47, 48]
  [ 49, 50, 51, 52, 53, 54, 55, 56]
  [ 57, 58, 59, 60, 61, 62, 63, 64]
]
[
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
]
[
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
  [ 0, 0, 0, 0, 0, 0, 0, 0]
]
]

```

FIGURE 2.6 – Détection et correction de l'erreur avec GDB - suite.

```

Program received signal SIGSEGV, Segmentation fault.
0x0000555555555bfb in ssgemm (m=8, n=8, k=8, alpha=1, A=0x55555556aeb0, B=0x55555556b080, beta=1, C=0x55555556b250) at src/cblasl3.cc:
26
26      sum += A[i][p] * B[p][j];
(gdb) bt
#0  0x0000555555555bfb in ssgemm (m=8, n=8, k=8, alpha=1, A=0x55555556aeb0, B=0x55555556b080, beta=1, C=0x55555556b250)
    at src/cblasl3.cc:26
#1  0x0000555555555441 in main (argc=1, argv=0x7ffffffe3e8) at tests/test001.cc:48
(gdb) █

```

FIGURE 2.7 – Détection et correction de l'erreur avec GDB avec l'option bt.

Alors :

- L'erreur s'est produite dans la fonction `ssgemm()` à la ligne 26 du fichier `cblasl3.cc`.
- Le programme a été appelé depuis la fonction `main()` à la ligne 48 du fichier `test001.cc`.

Ajout d'un point d'arrêt dans la fonction `ssgemm` pour le débogage

```

(gdb) print i
$1 = 8
(gdb) print j
$2 = 0
(gdb) print m
$3 = 8
(gdb) print n
$4 = 8

```

FIGURE 2.8 – Inspection des Variables.

Lors du débogage, l'analyse des valeurs des variables a révélé une incohérence dans l'indexation :

- La variable i a une valeur de 8, alors que la dimension m est également 8.
- Or, en programmation avec des indices basés sur 0, les indices valides vont de 0 à $m - 1$, soit 0 à 7 dans ce cas.

La valeur de $i = 8$ implique que l'index dépasse la limite $m - 1$, ce qui entraîne un **dépassement de tableau** (*out-of-bounds access*) dans $A[i]$.

Vérification de l'Accès Mémoire

L'accès à $A[i][p]$ est invalide à $i = 8$. Cela confirme que i dépasse les limites de la matrice allouée.

```
(gdb) print &A[i][p]
$5 = (float *) 0x0
(gdb) print &B[p][j]
$6 = (float *) 0x55555556b0d0
(gdb) print A[i][p]
Cannot access memory at address 0x0
```

FIGURE 2.9 – Vérification de l'Accès Mémoire.

Modification et Recompilation et test : (ligne 18 et 20)

```
1 //cblas3.cc
2
3 #include <cblas3.hh>
4 #include <iostream>
5 /*
6  * Routines of cblas level 3
7  */
8
9 void ssgemm(
10     int m, int n, int k,
11     float alpha,
12     float ** A,
13     float ** B,
14     float beta,
15     float ** C)
16 {
17     for (int i = 0; i < m; ++i) { // i<m not i<=m
18
19         for (int j = 0; j < n; ++j) { // j < n not j <= n
20
21             float sum = 0.0f;
22
23             for (int p = 0; p < k; ++p) {
24                 sum += A[i][p] * B[p][j];
25             }
26
27             C[i][j] = alpha * sum + beta * C[i][j];
28         }
29     }
30 }
31
32 // std::cout << "*****" << std::endl;
33 // std::cout << "  WARNING: Not Yet Implemented..." << std::endl;
34 // std::cout << "*****" << std::endl;
35 }
```

FIGURE 2.10 – Modification et Recompilation et test : (ligne 18 et 20).

```

=====
Matrices after computations :
=====
[
[ 1, 2, 3, 4, 5, 6, 7, 8]
[ 9, 10, 11, 12, 13, 14, 15, 16]
[ 17, 18, 19, 20, 21, 22, 23, 24]
[ 25, 26, 27, 28, 29, 30, 31, 32]
[ 33, 34, 35, 36, 37, 38, 39, 40]
[ 41, 42, 43, 44, 45, 46, 47, 48]
[ 49, 50, 51, 52, 53, 54, 55, 56]
[ 57, 58, 59, 60, 61, 62, 63, 64]
]
[
[ 1, 1, 1, 1, 1, 1, 1, 1]
[ 1, 1, 1, 1, 1, 1, 1, 1]
[ 1, 1, 1, 1, 1, 1, 1, 1]
[ 1, 1, 1, 1, 1, 1, 1, 1]
[ 1, 1, 1, 1, 1, 1, 1, 1]
[ 1, 1, 1, 1, 1, 1, 1, 1]
[ 1, 1, 1, 1, 1, 1, 1, 1]
[ 1, 1, 1, 1, 1, 1, 1, 1]
]
[
[ 36, 36, 36, 36, 36, 36, 36, 36]
[ 100, 100, 100, 100, 100, 100, 100, 100]
[ 164, 164, 164, 164, 164, 164, 164, 164]
[ 228, 228, 228, 228, 228, 228, 228, 228]
[ 292, 292, 292, 292, 292, 292, 292, 292]
[ 356, 356, 356, 356, 356, 356, 356, 356]
[ 420, 420, 420, 420, 420, 420, 420, 420]
[ 484, 484, 484, 484, 484, 484, 484, 484]
]

```

FIGURE 2.11 – Modification et Recompilation et test : (ligne 18 et 20) - suite.

Correction Apportée

- La correction dans la boucle `for (int i = 0; i <= m; ++i)` et `for (int j = 0; j <= n; ++j)` a permis d'éviter les erreurs de dépassement d'indice, comme je l'ai détecté précédemment.

- **Résultats :** La sortie montre les matrices avant et après les calculs, et la matrice **C** est calculée comme prévu.

Exécution sous Valgrind

Valgrind est un outil essentiel dans le processus de débogage et d'optimisation des programmes. Son objectif principal est d'aider à détecter des problèmes liés à la gestion de la mémoire, qui peuvent être difficiles à repérer par d'autres moyens.

Pourquoi utiliser Valgrind :

- **Détection des fuites de mémoire** qui peuvent mener à des consommations excessives de ressources.

- **Détection des erreurs d'accès à la mémoire**, comme les accès à des zones invalides ou hors limites.
- **Vérification de la validité des accès mémoire** pour garantir un comportement correct du programme.
- **Optimisation de la gestion de la mémoire** pour améliorer les performances globales.
- **Facilite le débogage et rend le processus de développement plus efficace.**

Assurons que notre programme est correctement compilé avec les symboles de débogage :

```
adel@adel:~/profiling/prflib001/prflast/prflib001$ g++ -g -Wall -I./include -std=c++17 -o bin/test001.exe src/cblasl0.cc src/cblasl1.cc src/cblasl3.cc tests/test001.cc
adel@adel:~/profiling/prflib001/prflast/prflib001$ ^C
```

FIGURE 2.12 – Compilation avec les symboles de débogage.

La commande a fonctionné correctement. Il n'y a pas de message d'erreur dans la sortie, donc la compilation a été réalisée avec succès, et l'exécutable bin/test001.exe a été généré sans problème.

Lancer Valgrind sur le programme

AVEC : `valgrind --leak-check=full --track-origins=yes --show-leak-kinds=all --verbose ./bin/test001.exe`

```
==3867==
==3867== HEAP SUMMARY:
==3867==    in use at exit: 0 bytes in 0 blocks
==3867== total heap usage: 29 allocs, 29 frees, 74,688 bytes allocated
==3867==
==3867== All heap blocks were freed -- no leaks are possible
==3867==
==3867== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

FIGURE 2.13 – Lancement du Valgrind sur le programme.

-All heap blocks were freed – no leaks are possible signifie qu'il n'y a pas de fuites de mémoire dans notre programme. -total heap usage : 29 allocs, 29 frees montre que 29 blocs de mémoire ont été alloués et tous ont été correctement libérés à la fin de l'exécution. Cela signifie qu'il n'y a aucune fuite de mémoire. -ERROR SUMMARY : 0 errors indique qu'il n'y a pas eu d'erreurs liées à la mémoire, telles que des accès à de la mémoire non allouée ou des erreurs de segmentation. Donc : Le programme ne présente aucune fuite de mémoire et aucune erreur de gestion de la mémoire n'a été détectée par Valgrind.

2.3 Conclusion

À travers cet exercice, j'ai pu implémenter la fonction `ssgemv` et utiliser des outils de débogage pour identifier et corriger les erreurs éventuelles.

Grâce aux tests et aux corrections apportées, le programme est désormais stable et prêt pour les prochaines étapes d'analyse de performance et d'optimisation. Dans ce qui suit, je présenterai la solution de l'exercice 3.

Exercice 3. Mesure de performance

3.1 Introduction

Une fois le programme fonctionnel, il est essentiel d'évaluer ses performances afin d'identifier d'éventuels goulets d'étranglement et d'optimiser son exécution. Cet exercice porte sur l'utilisation de différentes méthodes pour mesurer le temps d'exécution et le nombre de cycles processeur nécessaires à l'exécution du programme.

Dans cet exercice, je vais mesurer les performances d'une fonction de multiplication de matrices en utilisant la commande `time` et la fonction `rdtsc`. **Pour une analyse plus approfondie, je vais exécuter cet exercice dans deux environnements distincts :**

1. **Une machine virtuelle** : Initialement, j'ai effectué les tests dans un environnement virtualisé, où les performances peuvent être influencées par les ressources allouées à la machine hôte, la virtualisation et la gestion des cycles CPU.
2. **Une installation native d'Ubuntu** : Afin d'obtenir une comparaison plus représentative, je vais refaire l'exercice sur une installation isolée d'Ubuntu, sans l'impact des limitations d'une machine virtuelle.

L'objectif de cette comparaison est d'analyser comment l'environnement d'exécution affecte les performances du programme, notamment en termes de temps d'exécution.

3.2 Commenant par la machine virtuelle

Étapes à suivre pour utiliser la commande `time` :

Exécution du programme avec time : `time ./bin/test001.exe`

```

=====
Matrices after computations :
=====
[
  [ 1, 2, 3, 4, 5, 6, 7, 8]
  [ 9, 10, 11, 12, 13, 14, 15, 16]
  [ 17, 18, 19, 20, 21, 22, 23, 24]
  [ 25, 26, 27, 28, 29, 30, 31, 32]
  [ 33, 34, 35, 36, 37, 38, 39, 40]
  [ 41, 42, 43, 44, 45, 46, 47, 48]
  [ 49, 50, 51, 52, 53, 54, 55, 56]
  [ 57, 58, 59, 60, 61, 62, 63, 64]
]
[
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
]
[
  [ 36, 36, 36, 36, 36, 36, 36, 36]
  [ 100, 100, 100, 100, 100, 100, 100, 100]
  [ 164, 164, 164, 164, 164, 164, 164, 164]
  [ 228, 228, 228, 228, 228, 228, 228, 228]
  [ 292, 292, 292, 292, 292, 292, 292, 292]
  [ 356, 356, 356, 356, 356, 356, 356, 356]
  [ 420, 420, 420, 420, 420, 420, 420, 420]
  [ 484, 484, 484, 484, 484, 484, 484, 484]
]
real    0m0,020s
user    0m0,008s
sys     0m0,010s

```

FIGURE 3.1 – Exécution du programme avec time : `time ./bin/test001.exe`.

Lorsque j'ai exécuté cette commande, `time` a affiché trois informations principales :

- **real** : Le temps total écoulé depuis que la commande a été lancée.
- **user** : Le temps CPU que le programme a passé à exécuter du code en mode utilisateur.
- **sys** : Le temps CPU que le programme a passé dans le noyau pour des appels systèmes.

1. **real (Temps écoulé total) : 0m0,020s**

- Cela signifie que le **temps total** écoulé entre le lancement et la fin de l'exécution est de **20 millisecondes**. C'est le temps global qu'on attend pendant que le programme s'exécute, y compris le temps de traitement du programme et toute attente d'I/O (entrée/sortie).

2. **user (Temps CPU en mode utilisateur) : 0m0,008s**

- Le **temps utilisateur** de **8 millisecondes** signifie que le programme a passé ce temps à exécuter des instructions directement liées à la logique du programme (calculs,

manipulations de données, etc.). C'est le temps où le CPU était pleinement utilisé pour exécuter le programme sans interagir avec le système.

3. sys (Temps CPU en mode noyau) : 0m0,010s

- Le **temps système** de **10 millisecondes** indique que le programme a passé ce temps à effectuer des opérations systèmes, telles que des appels système pour gérer les fichiers, la mémoire, ou d'autres ressources externes. C'est du temps utilisé par le noyau pour gérer des ressources nécessaires à l'exécution du programme.

Interprétation :

- Le temps **real** est **20 millisecondes**, ce qui est relativement court. Cela signifie que notre programme est exécuté très rapidement.
- Le temps **user** et **sys** totalisent **18 millisecondes** (**8 ms** en mode utilisateur et **10 ms** en mode noyau), ce qui montre que le programme a passé la majeure partie de son temps à exécuter des instructions logiques et à interagir avec le système.

La fonction rdtsc :

- **La fonction rdtsc** (Read Time-Stamp Counter) permet de lire le compteur de temps de l'horloge du processeur, qui est incrémenté en cycles d'horloge. Cela permet de mesurer le nombre de **cycles d'exécution** d'un programme, ce qui peut être très utile pour analyser la performance de certaines fonctions à un niveau très précis.
- L'objectif ici est d'utiliser **rdtsc** pour mesurer précisément le nombre de cycles d'exécution de la fonction **ssgemv** (qui est un produit matriciel).

Modification du code de ssgemm pour ajouter la mesure avec rdtsc

```

C: > Users > adelb > Documents > debo_profil_c > prflib001 > prflast > prflib001 > src > cblas3.cc
1  #include <iostream>
2  #include <inttypes> // ==> Pour uint64_t
3
4  inline uint64_t rdtsc() {
5      uint32_t lo, hi;
6      __asm__ volatile("rdtsc" : "=a"(lo), "=d"(hi));
7      return ((uint64_t)hi << 32) | lo;
8  }
9
10 void ssgemm(int m, int n, int k, float alpha, float **A, float **B, float beta, float **C) {
11     uint64_t start_time = rdtsc(); // mesure avant l'exécution
12
13     for (int i = 0; i < m; ++i) {
14         for (int j = 0; j < n; ++j) {
15             float sum = 0.0f;
16             for (int p = 0; p < k; ++p) {
17                 sum += A[i][p] * B[p][j];
18             }
19             C[i][j] = alpha * sum + beta * C[i][j];
20         }
21     }
22
23     uint64_t end_time = rdtsc(); // mesurer après l'exécution
24     uint64_t cycles = end_time - start_time; // calcul du nombre de cycles
25     std::cout << "cycles d'exécution de ssgemm: " << cycles << std::endl;
26 }

```

FIGURE 3.2 – Modification du code de ssgemm pour ajouter la mesure avec rdtsc.

Compilation et exécution du programme

make clean make ./bin/test001.exe

```

Cycles d'exécution de ssgemm: 9140
=====
Matrices after computations :
=====
[
  [ 1, 2, 3, 4, 5, 6, 7, 8]
  [ 9, 10, 11, 12, 13, 14, 15, 16]
  [ 17, 18, 19, 20, 21, 22, 23, 24]
  [ 25, 26, 27, 28, 29, 30, 31, 32]
  [ 33, 34, 35, 36, 37, 38, 39, 40]
  [ 41, 42, 43, 44, 45, 46, 47, 48]
  [ 49, 50, 51, 52, 53, 54, 55, 56]
  [ 57, 58, 59, 60, 61, 62, 63, 64]
]
[
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
  [ 1, 1, 1, 1, 1, 1, 1, 1]
]
[
  [ 36, 36, 36, 36, 36, 36, 36, 36]
  [ 100, 100, 100, 100, 100, 100, 100, 100]
  [ 164, 164, 164, 164, 164, 164, 164, 164]
  [ 228, 228, 228, 228, 228, 228, 228, 228]
  [ 292, 292, 292, 292, 292, 292, 292, 292]
  [ 356, 356, 356, 356, 356, 356, 356, 356]
  [ 420, 420, 420, 420, 420, 420, 420, 420]
  [ 484, 484, 484, 484, 484, 484, 484, 484]
]

```

FIGURE 3.3 – Compilation et exécution du programme.

Le résultat qu'on a obtenu avec rdtsc montre que la fonction ssgemm a été exécutée avec

succès et le nombre de cycles d'exécution est indiqué comme 9140. Le résultat de 9140 cycles donne une idée de la performance de cette multiplication de matrices. Cela peut être utile pour comparer l'efficacité du programme avec différentes tailles de matrices ou pour optimiser des algorithmes similaires.

3.2.1 Refaire l'Exercice 3 sur la nouvelle installation Ubuntu

Installation des outils nécessaires

```
sudo apt update sudo apt install build-essential g++ time
```

Compilation du code source :

```
adel@adel:~/Documents/prflib001$ make clean
rm -f ./build/*.o ./lib/*.a ./bin/*.exe src/*~ include/*~
adel@adel:~/Documents/prflib001$ make
g++ -Wall -I./include -std=c++17 -g src/cblasl0.cc -c -o build/cblasl0.o
g++ -Wall -I./include -std=c++17 -g src/cblasl1.cc -c -o build/cblasl1.o
g++ -Wall -I./include -std=c++17 -g src/cblasl3.cc -c -o build/cblasl3.o
ar -rc lib/prflib.a build/cblasl0.o build/cblasl1.o build/cblasl3.o
g++ -Wall -I./include -std=c++17 -g tests/test001.cc -c -o build/test001.o
g++ -Wall -I./include -std=c++17 -g build/test001.o lib/prflib.a -o bin/test001.exe
```

FIGURE 3.4 – Compilation du code source.

Exécution du programme avec time : (time ./bin/test001.exe)

```
real    0m0,007s
user    0m0,003s
sys     0m0,004s
```

FIGURE 3.5 – Exécution du programme avec time : (time ./bin/test001.exe).

Exécution du programme avec rdtsc pour mesurer les cycles CPU : (./bin/test001.exe)

```
]
Cycles d'exécution de ssgemm: 6466
```

FIGURE 3.6 – Exécution du programme avec rdtsc pour mesurer les cycles CPU : (./bin/test001.exe).

Analyse comparative des performances : Machine Virtuelle vs Installation Ubuntu Native

Environnement	real (s)	user (s)	sys (s)	Cycles d'exécution
Machine Virtuelle	0,020	0,008	0,010	9140
Ubuntu (Installation native)	0,007	0,003	0,004	6466

TABLE 3.1 – Comparaison des performances entre une machine virtuelle et une installation native.

Interprétation

Ubuntu installé en natif est nettement plus performant que la machine virtuelle :

- 108 Temps d'exécution **réduit de 65%** (real).
- 108 Moins de cycles d'exécution utilisés (**30% de cycles en moins**).
- 108 Moins de temps passé en mode noyau (**sys**) et utilisateur (**user**), indiquant une exécution plus fluide et optimisée.

Impact de la virtualisation :

- La VM introduit une surcharge qui allonge le temps d'exécution et augmente le nombre de cycles nécessaires.
- Les appels système sont significativement plus coûteux sur une VM.

3.3 Conclusion

Cet exercice m'a permis d'explorer différentes techniques de mesure de performance, notamment à travers l'utilisation des commandes `time` et `rdtsc`. Grâce à ces outils, j'ai pu quantifier le temps d'exécution du programme et analyser le nombre de cycles processeur consommés, fournissant ainsi des indicateurs clés sur son efficacité.

Dans ce qui suit, je présenterai la solution de l'exercice 4.

Exercice 4. Profiling simple

4.1 Introduction

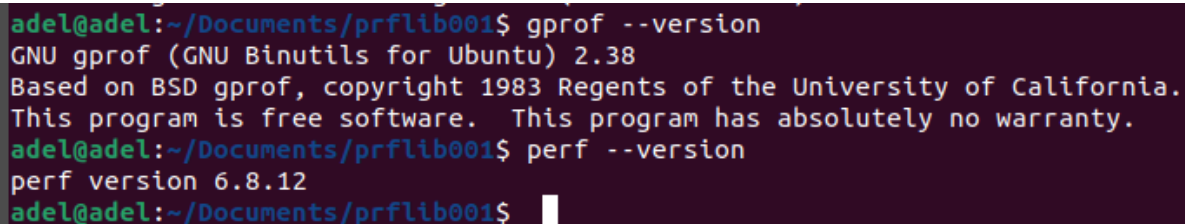
Dans cet exercice, je vais effectuer un profiling simple de notre programme en utilisant les outils gprof et perf. Contrairement à l'exercice précédent, qui a été réalisé sur deux environnements distincts (machine virtuelle et installation native d'Ubuntu), cet exercice sera exclusivement réalisé sur l'installation native d'Ubuntu.

L'objectif est d'analyser le comportement du programme en termes d'exécution et de consommation de ressources sans les contraintes imposées par la virtualisation. En effectuant le profilage directement sur une installation native, on obtient des résultats plus précis et représentatifs des performances réelles de notre application.

je vais donc :

- 108 **Utiliser gprof** pour obtenir une première analyse des parties les plus coûteuses en temps d'exécution.
- 108 **Utiliser perf** pour mesurer plus précisément le nombre de cycles CPU et d'instructions exécutées.

Les outils nécessaires :



```
adel@adel:~/Documents/prflib001$ gprof --version
GNU gprof (GNU Binutils for Ubuntu) 2.38
Based on BSD gprof, copyright 1983 Regents of the University of California.
This program is free software. This program has absolutely no warranty.
adel@adel:~/Documents/prflib001$ perf --version
perf version 6.8.12
adel@adel:~/Documents/prflib001$
```

FIGURE 4.1 – Les outils nécessaires.

Étape 1 : Profiling avec gprof (Q-4.1)

Pour activer le profilage, **recompile le programme** avec l'option `-pg` :

```
make clean
g++ -pg -g -Wall -I./include -std=c++17 -o bin/test001.exe
src/cblas0.cc src/cblas1.cc src/cblas3.cc tests/test001.cc
```

Exécute ensuite le programme normalement :

```
./bin/test001.exe
```

Analyse du profil avec gprof :

```
gprof ./bin/test001.exe gmon.out > gprof_report.txt
cat gprof_report.txt
```

Les résultats obtenus avec gprof - Le "Flat Profile"

```
root@adel:/home/adel/Documents/prflib001# cat gprof_report.txt
Flat profile:

Each sample counts as 0.01 seconds.
no time accumulated

%   cumulative   self           self         total
time  seconds    seconds   calls   Ts/call   Ts/call   name
0.00      0.00      0.00         6      0.00      0.00  mmsdump(int, int, float**)
0.00      0.00      0.00         3      0.00      0.00  mmsfree(float**, int, int)
0.00      0.00      0.00         3      0.00      0.00  mmsinit(int, int, float**, float, float)
0.00      0.00      0.00         3      0.00      0.00  mmsalloc(int, int)
0.00      0.00      0.00         2      0.00      0.00  rdtsc()
0.00      0.00      0.00         1      0.00      0.00  __static_initialization_and_destruction_0(int, int)
0.00      0.00      0.00         1      0.00      0.00  __static_initialization_and_destruction_0(int, int)
0.00      0.00      0.00         1      0.00      0.00  __static_initialization_and_destruction_0(int, int)
0.00      0.00      0.00         1      0.00      0.00  ssgemm(int, int, int, float, float**, float**, float, float**)
```

FIGURE 4.2 – Les résultats obtenus avec gprof - Le "Flat Profile".

Les résultats obtenus avec gprof -Le "Call Graph"

granularity: each sample hit covers 4 byte(s) no time propagated

index	% time	self	children	called	name
[8]	0.0	0.00	0.00	6/6	main [6]
		0.00	0.00	6	mmsdump(int, int, float**) [8]
[9]	0.0	0.00	0.00	3/3	main [6]
		0.00	0.00	3	mmsfree(float**, int, int) [9]
[10]	0.0	0.00	0.00	3/3	main [6]
		0.00	0.00	3	mmsinit(int, int, float**, float, float) [10]
[11]	0.0	0.00	0.00	3/3	main [6]
		0.00	0.00	3	mmsalloc(int, int) [11]
[12]	0.0	0.00	0.00	2/2	ssgemm(int, int, int, float, float**, float**, float, float**) [16]
		0.00	0.00	2	rdtsc() [12]
[13]	0.0	0.00	0.00	1/1	_GLOBAL__sub_I__Z6salloca [17]
		0.00	0.00	1	__static_initialization_and_destruction_0(int, int) [13]
[14]	0.0	0.00	0.00	1/1	_GLOBAL__sub_I__Z6ssgemmiifPPf50_fs0 [18]
		0.00	0.00	1	__static_initialization_and_destruction_0(int, int) [14]
[15]	0.0	0.00	0.00	1/1	_GLOBAL__sub_I_main [19]
		0.00	0.00	1	__static_initialization_and_destruction_0(int, int) [15]
[16]	0.0	0.00	0.00	1/1	main [6]
		0.00	0.00	1	ssgemm(int, int, int, float, float**, float**, float, float**) [16]
		0.00	0.00	2/2	rdtsc() [12]

FIGURE 4.3 – Les résultats obtenus avec gprof - Le "Call Graph".

4.1.1 Résumé et interprétation des résultats gprof - Observations principales

Temps d'exécution total (Flat Profile) :

- 108 Toutes les fonctions ont **un temps d'exécution proches de 0.00 secondes**.
- 108 `ssgemm()`, qui est censée être une fonction clé pour la multiplication de matrices, **n'affiche aucun temps mesuré**.
- 108 **Aucune accumulation de temps CPU** → gprof ne capture pas assez d'échantillons.

Nombre d'appels :

- 108 `ssgemm()` n'a été appelée **qu'une seule fois**.
- 108 `rdtsc()` a été exécutée **deux fois**, ce qui correspond à sa fonction de mesure avant et après l'exécution du programme.
- 108 D'autres fonctions comme `mmsdump()`, `mmsinit()`, `mmsalloc()` et `mmsfree()` ont été appelées quelques fois, mais leur impact reste négligeable.

Call Graph (Arbre d'appels) :

- `main()` appelle plusieurs fonctions mais aucune ne montre un impact mesurable sur le temps CPU.
- `ssgemm()` est bien listée, mais aucun temps d'exécution ne lui est attribué.
- `rdtsc()` est appelée par `ssgemm()`, mais comme son exécution est instantanée, elle n'affecte pas les résultats.

4.1.2 Analysons les résultats

Pourquoi gprof n’affiche-t-il aucun temps ?

- 108 Le **programme s’exécute trop vite** pour que gprof puisse capturer un échantillonnage significatif.
- 108 gprof fonctionne par **échantillonnage**, donc si un programme est trop court, il ne collecte **aucune donnée pertinente**.
- 108 `ssgemm()` est **appelée une seule fois**, donc l’analyse de son impact est **très limitée**.

Pour obtenir des statistiques détaillées et significatives sur l’exécution du programme, plusieurs modifications ont *été* appliquées :

1. Augmentation de la Taille des Matrices

- 108 Initialement, les tests étaient effectués sur des matrices de **taille 8×8**.
- 108 Afin de **prolonger le temps d’exécution** et de collecter suffisamment de données pour gprof, j’ai progressivement **augmenté la taille des matrices**.
- 108 Cette augmentation garantit un **temps de calcul suffisant pour être mesuré avec précision** et permet d’observer l’impact sur l’utilisation du CPU.

Modification apportée sur le script `test001.cc` : (remplacer 8 par 252)

```
17 int main(int argc, char **argv)
18 {
19
20     int Size = 252; // Valeur par défaut
21
22     if (argc > 1) {
23         Size = atoi(argv[1]); // Mise à jour si un argument est passé
24     }
25
26     // Création des vecteurs
```

FIGURE 4.4 – Modification apportée sur le script `test001.cc` : (remplacer 8 par 252).

2. Modification du Code pour Permettre un Profilage Efficace

- 108 Suppression de la fonction `rdtsc()` (qui mesurait les cycles CPU), car gprof fournit déjà une analyse détaillée du temps d’exécution.

Nouveau Cblasl3 :

```

1 #include <iostream>
2
3 /**
4  * Fonction de multiplication de matrices SSGEMM
5  * C = beta * C + alpha * (A * B)
6  * A (m, k)
7  * B (k, n)
8  * C (m, n)
9  */
10 void ssgemm(int m, int n, int k, float alpha, float **A, float **B, float beta, float **C) {
11
12     // Répéter 1000 fois pour forcer un temps d'exécution plus long
13     for (int rep = 0; rep < 1000; ++rep) {
14         for (int i = 0; i < m; ++i) {
15             for (int j = 0; j < n; ++j) {
16                 float sum = 0.0f;
17                 for (int p = 0; p < k; ++p) {
18                     sum += A[i][p] * B[p][j];
19                 }
20                 C[i][j] = alpha * sum + beta * C[i][j];
21             }
22         }
23     }
24
25     // Affichage pour vérifier que l'exécution a bien eu lieu
26     std::cout << "Multiplication terminée pour une matrice de " << m << "x" << n << " exécutée 1000 fois." << std::endl;
27 }

```

FIGURE 4.5 – Modification du Code pour Permettre un Profilage Efficace.

Recompile le programme avec l'option -pg et exécuter

Analyse du profil avec gprof :

```

gprof ./bin/test001.exe gmon.out >
gprof_report_avec_nouvelle_taille_de_matrices.txt
cat gprof_report_avec_nouvelle_taille_de_matrices.txt

```

Recompilation du programme avec l'option -pg et exécution - Le "Flat Profile"

```

root@adel:/home/adel/Documents/prflib001# gprof ./bin/test001.exe gmon.out > gprof_report_avec_nouvelle_taille_de_matrices.txt
root@adel:/home/adel/Documents/prflib001# cat gprof_report_avec_nouvelle_taille_de_matrices.txt
Flat profile:

Each sample counts as 0.01 seconds.
 %   cumulative   self           calls     self        total   name
time  seconds    seconds               s/call       s/call       name
100.00      63.39      63.39              1      63.39      63.39  ssgemm(int, int, int, float, float**, float**, float, float**)
 0.00      63.39      0.00               3       0.00       0.00  mmsfree(float**, int, int)
 0.00      63.39      0.00               3       0.00       0.00  mmsinit(int, int, float**, float, float)
 0.00      63.39      0.00               3       0.00       0.00  mmsalloc(int, int)
 0.00      63.39      0.00               1       0.00       0.00  __static_initialization_and_destruction_0(int, int)
 0.00      63.39      0.00               1       0.00       0.00  __static_initialization_and_destruction_0(int, int)
 0.00      63.39      0.00               1       0.00       0.00  __static_initialization_and_destruction_0(int, int)

```

FIGURE 4.6 – Recompilation du programme avec l'option -pg et exécution - Le "Flat Profile".

Recompilation du programme avec l'option -pg et exécution - Le "Call Graph"

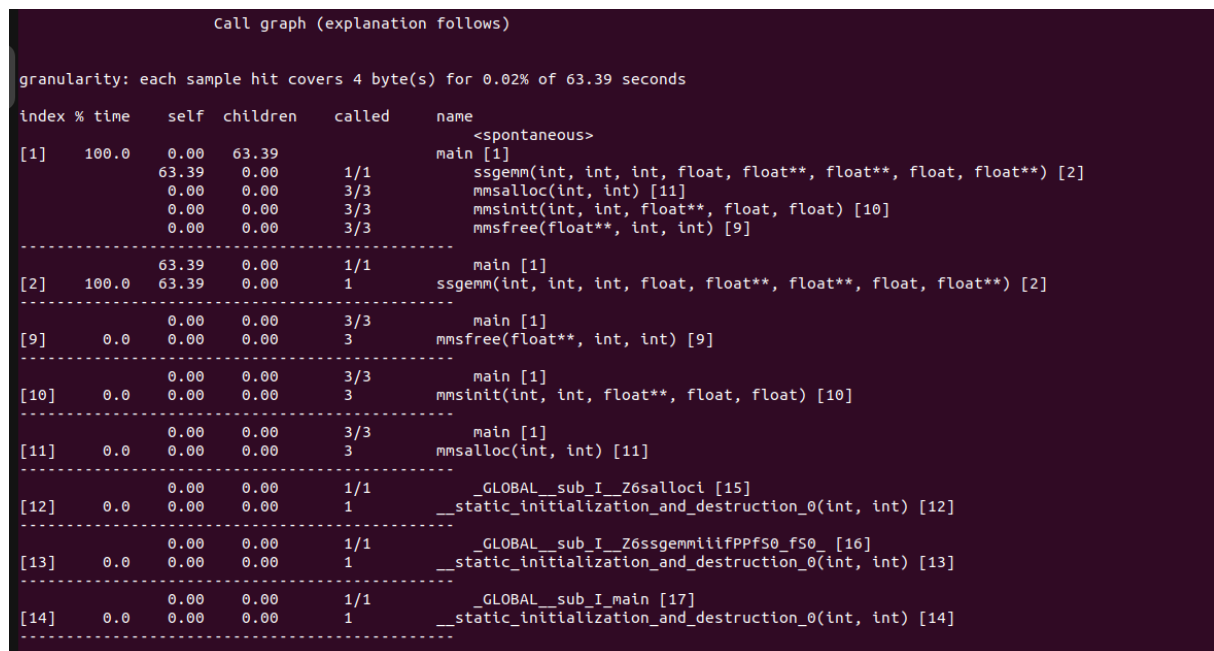


FIGURE 4.7 – Recompilation du programme avec l'option -pg et exécution - Le "Call Graph".

Après l'augmentation de la taille des matrices et l'exécution du profilage avec gprof, j'ai obtenu les résultats suivants. Cette analyse me permettra d'évaluer l'impact des modifications et de mieux comprendre l'utilisation des ressources CPU.

Temps d'Exécution Total :

- 108 Temps total d'exécution : 63.39 secondes
- 108 Temps consommé par ssgemm : 100% (63.39 secondes)
- 108 Temps consommé par les autres fonctions (mmsalloc, mmsinit, mmsfree) : 0.00%

Interprétation :

La quasi-totalité du temps d'exécution est absorbée par la multiplication de matrices ssgemm, confirmant que cette fonction est le principal goulot d'étranglement. Les fonctions d'allocation et de libération mémoire (mmsalloc, mmsinit, mmsfree) ont un impact négligeable sur les performances.

Étape 2 : L'outil perf :

Exécution et Analyse avec perf

Je vais exécuter le programme et collecter des statistiques sur le nombre de cycles CPU et instructions exécutées.

```

root@adel:/home/adel/Documents/prflib001# perf stat -e cycles,instructions ./bin/test001.exe
Multiplication terminée pour une matrice de 252x252 exécutée 1000 fois.

Performance counter stats for './bin/test001.exe':

   206 090 857 571      cycles
   482 912 669 008      instructions          #    2,34  insn per cycle

   69,708306544 seconds time elapsed

   69,402156000 seconds user
    0,043978000 seconds sys

```

FIGURE 4.8 – Exécution et Analyse avec perf.

1) Lancer le programme avec perf :

Analyse des Résultats de perf :

Métrique	Valeur	Explication
Cycles CPU	206 090 857 571 cycles	Nombre total de cycles processeur consommés.
Instructions exécutées	482 912 669 008 instructions	Nombre total d'instructions exécutées.
Ratio Instructions/Cycle	2,34 insn per cycle	En moyenne, 2.34 instructions exécutées par cycle CPU.
Temps total	69,71 s	Temps total d'exécution du programme.
Temps utilisateur (user)	69,40 s	Temps passé en mode utilisateur (exécution du programme lui-même).
Temps système (sys)	0,04 s	Temps passé en mode kernel (gestion mémoire, I/O, etc.).

TABLE 4.1 – Statistiques d'exécution du programme

Temps d'exécution total (69,71 s) :

- 108 Principalement passé en **mode utilisateur** (69,40 s), ce qui est attendu pour un programme de calcul intensif.
- 108 Très peu de temps en **mode système** (0,04 s), ce qui signifie qu'il n'y a pas beaucoup d'opérations d'entrée/sortie ou d'appels système.

Comparaison avec gprof :

Dans gprof, j'ai observé :

Métrique	Valeur avec gprof	Valeur avec perf
Temps total	63,39 s	69,71 s
Temps dans ssgemm	100% (63,39s)	100% (69,71s)
Nombre d'appels à ssgemm	1	1
Fonctions secondaires (mmsalloc, mmsfree)	Négligeables	Négligeables
Autres fonctions (main, mmsinit)	Non significatif	Non significatif

TABLE 4.2 – Comparaison des valeurs avec gprof et perf

Observations :

- **Temps similaire** entre gprof (63,39 s) et perf (69,71 s), ce qui confirme la cohérence des mesures.
- **ssgemm() représente 100% du temps d'exécution**, ce qui signifie qu'il s'agit du **point chaud (hotspot) principal** du programme.
- **perf fournit plus d'informations sur les cycles et instructions**, tandis que gprof montre uniquement la répartition des appels.

4.2 Conclusion

Cet exercice m'a permis de découvrir l'importance du profiling dans l'optimisation des performances d'un programme. Grâce à l'utilisation de gprof et perf, j'ai pu identifier les fonctions les plus coûteuses en termes de temps d'exécution et de cycles processeur.

Dans ce qui suit, je présenterai la conclusion de ce travail.

Conclusion Générale

Ce TD m'a permis d'explorer les principales étapes du développement et de l'optimisation d'un programme. À travers l'installation du code source, l'implémentation de la fonction `ssgemv`, et l'utilisation d'outils de débogage et de profiling.

L'identification et la correction des erreurs via des outils comme GDB permettent d'améliorer la robustesse du code, tandis que l'analyse des performances à l'aide de `time`, `rdtsc`, `gprof` et `perf` m'a offert une meilleure compréhension de l'impact des choix d'implémentation sur l'exécution du programme. Ces outils sont indispensables pour tout développeur souhaitant détecter les goulots d'étranglement, réduire la consommation de ressources et optimiser les performances d'un programme.